

SIMATS ENGINEERING

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	KAVALI BALAJI	Reg.No.:	192525402
Department:	AI&ML	Date:	25-08-2026

ANNEXURE A

INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
2. Cloud Server Memory Bottleneck
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.
3. Autonomous Vehicle Data Processing
An autonomous vehicle must process sensor data in real time. Evaluate SRAM,

DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

5. High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct:

I KAVALI BALAJI(192525402)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and	Good analysis with	Basic analysis	Poor or no analysis

		performance evaluation	reasonable validation	with limited validation	
--	--	---------------------------	--------------------------	----------------------------	--

1. Smartphone Performance Optimization

a) Problem Understanding

A modern smartphone is expected to run many applications simultaneously while providing a smooth and responsive user experience. Applications such as browsers, social media, video streaming, gaming, navigation and messaging continuously generate processor and memory requests. When the user switches between applications, the processor must retrieve instructions and data belonging to the selected application. If this information is not available in the faster cache levels, the processor has to access DRAM, increasing latency.

The industry problem is therefore to redesign the memory hierarchy so that frequently accessed information remains close to the processor, large working sets are available in high-bandwidth DRAM, and unnecessary memory movement is reduced. The solution must improve latency and throughput without causing excessive power consumption.

b) Industry Scenario

Consider a smartphone with a multicore processor running several applications. A user may move from a game to a browser, then to a messaging application and back to the game. If the game's working data has been displaced from cache, the processor must retrieve it again. Repeated cache misses can produce application reloads, delays and increased DRAM traffic.

This problem is especially important in mobile systems because smartphones have limited thermal and battery budgets. Simply increasing memory capacity does not necessarily solve latency. A balanced hierarchy is required.

c) Existing Memory Hierarchy

The typical hierarchy is:

CPU → L1 Cache → L2 Cache → L3 Cache → DRAM → Flash Storage

L1 is the smallest and fastest cache. L2 provides more capacity at slightly higher latency. L3 is generally larger and shared among cores. DRAM provides the large working memory required by the operating system and applications. Flash storage provides permanent data storage but is much slower than RAM.

d)Engineering Concepts

The proposed design uses temporal locality and spatial locality. Temporal locality means recently accessed data is likely to be accessed again. Spatial locality means nearby addresses are likely to be accessed soon. Cache memory exploits these properties by keeping useful blocks close to the CPU.

Prefetching can also be used to load data before it is requested. Cache-aware scheduling can keep important working sets available to the appropriate processor cores.

e)Proposed Solution

CPU → L1 → L2 → L3 → LPDDR5/LPDDR5X DRAM

The L1 cache should be optimized for immediate instructions and data. L2 should provide sufficient core-local capacity. A shared L3 cache can reduce communication and repeated DRAM accesses. High-speed LPDDR5/LPDDR5X DRAM should provide the main memory capacity while maintaining reasonable mobile power consumption.

f)Working

For every memory request, the processor first checks L1. If there is a miss, it checks L2, then L3, and finally DRAM. If the data is found at a higher level, it is supplied to the processor and may be copied into faster cache levels. This approach reduces the frequency of expensive DRAM accesses.

g)Performance Calculation

Assume L1 = 1 ns, L2 = 4 ns, L3 = 12 ns, DRAM = 80 ns, L1 hit rate = 90%, L2 hit rate after L1 miss = 80%, and L3 hit rate after L2 miss = 90%.

$$AMAT = 1 + (0.10 \times 4) + (0.10 \times 0.20 \times 12) + (0.10 \times 0.20 \times 0.10 \times 80)$$

$$AMAT = 1.80 \text{ ns.}$$

The simplified result demonstrates the benefit of a multilevel cache system.

h)Advantages

- Faster application switching
- Lower average memory latency
- Reduced CPU stalls
- Higher multitasking performance
- Lower DRAM traffic
- Better processor utilization

- Improved battery efficiency when memory movement is reduced

i)Conclusion

A smartphone should use an optimized L1–L2–L3 hierarchy backed by high-speed LPDDR5/LPDDR5X DRAM. Cache-aware scheduling, prefetching and locality optimization can reduce memory stalls and improve application responsiveness.

j)Smartphone Memory Comparison

Memory	Technology	Relative Speed	Capacity	Role
L1	SRAM	Extremely high	Very small	Immediate instructions/data
L2	SRAM	Very high	Small	Core working data
L3	SRAM	High	Larger	Shared data
DRAM	LPDDR5/LPDDR5X	High	Large	Applications/OS
Flash	NAND	Low	Very large	Permanent storage

k)Smartphone Proposed Design

Level	Data Stored	Optimization	Benefit
L1	Critical instructions/data	High hit rate	Minimum latency
L2	Core working set	Cache optimization	Fewer L3 accesses
L3	Shared application data	Prefetching	Fewer DRAM accesses
DRAM	Large application data	High bandwidth	Better throughput

2. Cloud Server Memory Bottleneck

a)Problem Understanding

A cloud service provider may experience high database query latency when processors spend excessive time waiting for data. Database workloads repeatedly access indexes, buffers, records and metadata. A cache miss forces access to lower memory levels, while a page fault may force the operating system to retrieve a page from storage. The objective is to reduce both processor-memory latency and expensive paging.

b)Industry Scenario

A cloud database can have thousands of concurrent users. If frequently accessed database pages are not retained in sufficient memory, the operating system may start paging. Storage access is many orders of magnitude slower than processor cache access, so even a small number of poorly managed page faults can increase response time.

c)Cache Hierarchy

CPU → L1 → L2 → L3 → DRAM

Cache hierarchy is designed to reduce latency. Frequently reused instructions and data can remain in cache, reducing accesses to DRAM.

d)Virtual Memory

Virtual memory provides applications with a logical address space larger than physical memory. Data is divided into pages. If a requested page is absent from DRAM, a page fault occurs and the operating system loads it from storage. This operation is much slower than cache or DRAM access.

e)Recommended Solution

CPU → L1 → L2 → L3 → DDR5 DRAM → NVMe SSD

The server should provide adequate DRAM to keep database working sets resident. L3 cache should be optimized, database indexes should be efficient, NUMA allocation should be considered, and NVMe storage should be used when storage access is unavoidable.

f)Performance Analysis

Assume cache access = 10 ns, DRAM = 100 ns and NVMe page-fault access $\approx 100 \mu\text{s}$. Since $100 \mu\text{s} = 100,000 \text{ ns}$, the simplified ratio is $100,000/10 = 10,000$. This illustrates why avoiding page faults is extremely important for latency-sensitive databases.

G)Tools

CPU performance counters, Linux perf, database monitoring, cache-miss counters, page-fault monitoring, DDR5 memory monitoring and NUMA analysis can be used.

h)Conclusion

Cache hierarchy and virtual memory address different requirements. Cache minimizes latency while virtual memory provides capacity and memory-management flexibility. Sufficient DRAM, efficient cache usage and fast NVMe storage provide a practical cloud-server solution.

i)Cloud Memory Technologies

Parameter	Cache	DRAM	Virtual Memory / NVMe
Purpose	Reduce CPU latency	Large working memory	Extend logical capacity
Technology	SRAM	DDR5 DRAM	SSD + DRAM
Latency	Very low	Higher than cache	Very high during page fault

Management	Hardware	Memory controller/OS	Operating system/MMU
Main Risk	Cache miss	Memory pressure	Page fault

j)Cloud Improvements

Problem	Proposed Improvement	Expected Effect
Cache misses	Optimize L3/cache locality	Lower CPU latency
Memory pressure	Increase DRAM	Fewer page faults
Storage delay	Use NVMe	Faster unavoidable storage access
NUMA delay	NUMA-aware allocation	Better multicore performance

3. Autonomous Vehicle Data Processing

a)Problem Understanding

An autonomous vehicle receives continuous data from cameras, LiDAR, radar, GPS and other sensors. The computing platform must process this information in real time for object detection, sensor fusion, path planning and control. Memory latency and bandwidth directly influence how quickly the vehicle can react.

b)Industry Scenario

A vehicle may simultaneously process high-resolution camera frames, LiDAR point clouds and radar measurements. These streams create a large memory bandwidth requirement. Critical control data cannot wait for slow storage operations, so time-sensitive information should be kept in very fast memory.

c)Technology Analysis

SRAM provides extremely low latency and is appropriate for caches and small working sets. DRAM provides much larger capacity and is appropriate for sensor buffers and AI inference data. MRAM provides non-volatility and can retain configuration or selected persistent information without refresh.

d)Proposed Hierarchy

CPU/AI Accelerator → SRAM → DRAM → MRAM

SRAM stores the most time-critical information. DRAM stores large sensor datasets and inference buffers. MRAM stores persistent parameters, configuration data and selected recovery information.

e)Bandwidth Calculation

Assume camera = 2 GB/s, LiDAR = 1 GB/s and radar = 0.5 GB/s.

Total = 2 + 1 + 0.5 = 3.5 GB/s.

The memory subsystem should therefore provide more than 3.5 GB/s and sufficient headroom for processing, additional sensors and future upgrades.

f)Real-Time Requirement

If a sensor frame arrives every 20 ms, critical processing should ideally finish within that interval. Memory transfer, sensor fusion, AI inference and decision logic must therefore be designed as a pipeline.

g)Advantages

- High real-time responsiveness
- Low latency for critical data
- High bandwidth for sensor streams
- Fast AI inference
- Persistent configuration through MRAM
- Reduced unnecessary memory transfers

h)Conclusion

A hybrid SRAM–DRAM–MRAM architecture provides a practical balance of latency, capacity, bandwidth and persistence for autonomous vehicles.

i)Autonomous Vehicle Memory Comparison

Feature	SRAM	DRAM	MRAM
Speed	Very high	High	High
Volatility	Volatile	Volatile	Non-volatile
Density	Low	High	Medium
Refresh	No	Yes	No
Best use	Cache/critical data	Sensor/AI buffers	Persistent fast data

j)Sensor Bandwidth Requirement

Sensor	Data Rate	Memory Requirement
Camera	2 GB/s	High continuous bandwidth
LiDAR	1 GB/s	High sequential/structured transfer
Radar	0.5 GB/s	Low-latency processing
Total	3.5 GB/s	Minimum combined requirement

4. AI Inference Edge Device

a) Problem Understanding

Edge AI devices execute machine-learning models close to the source of data. They are used in smart cameras, robots, industrial systems, drones and IoT devices. These systems have limited power and memory resources, while AI models may be large. The memory hierarchy must therefore load models quickly and avoid unnecessary energy consumption.

b) Industry Scenario

A smart camera may need to perform object detection continuously. The model can be stored permanently in NAND Flash, loaded into DRAM for active inference and supported by a fast persistent technology such as RRAM for frequently accessed information.

c) Technology Analysis

NAND Flash is non-volatile, high-capacity and suitable for long-term model storage. DRAM is volatile but provides high speed for active models and intermediate tensors. RRAM is non-volatile and can provide fast persistent storage for frequently accessed parameters.

d) Proposed Architecture

AI Accelerator → DRAM → RRAM → NAND Flash

NAND stores complete models. DRAM holds the active model and inference buffers. RRAM can retain frequently used parameters or persistent AI information.

e) Model Loading Calculation

Model size = 1 GB. NAND transfer rate = 1 GB/s. DRAM transfer rate = 20 GB/s.

NAND loading time = $1/1 = 1$ second.

DRAM transfer time = $1/20 = 0.05$ second = 50 ms.

This shows why fast working memory is important after the model is loaded.

f) Power Optimization

Inactive models should remain in NAND Flash. Only active working data should occupy DRAM. Power gating can be applied to unused memory blocks. Model compression and quantization can also reduce memory traffic.

g) Advantages

- Faster model loading

- Lower inference latency
- Lower energy use
- Local processing
- Reduced network dependency
- Better responsiveness

h)Conclusion

NAND Flash, DRAM and RRAM together provide a balanced hierarchy for edge AI. The architecture separates permanent storage from active computation and frequently accessed persistent information.

i)Edge AI Memory Comparison

Feature	NAND Flash	DRAM	RRAM
Volatility	Non-volatile	Volatile	Non-volatile
Speed	Medium/low	High	Potentially high
Capacity	High	Medium/high	Potentially high
Standby Power	Low	Higher	Low
Primary Role	Model repository	Active inference	Fast persistent data

j)Edge AI Data Placement

Memory	Data	Reason
NAND Flash	Complete AI models	High capacity and persistence
DRAM	Active model/tensors	Fast working memory
RRAM	Frequently used parameters	Fast non-volatile access

5. High-Performance Gaming Console

a)Problem Understanding

Modern games contain high-resolution textures, 3D models, animation data, shaders, audio and physics information. During scene loading, very large amounts of data must be transferred. If the CPU or GPU cannot obtain the required data quickly enough, stalls and frame drops can occur.

b)Industry Scenario

When a player moves rapidly through a large game environment, new textures and geometry must be streamed continuously. A memory system with insufficient bandwidth or poor cache utilization may cause stuttering. The objective is to provide low latency for frequently reused data and high bandwidth for large assets.

C)L3 Cache vs DRAM

L3 cache uses SRAM and provides low-latency access to frequently reused data. DRAM has much larger capacity and is responsible for storing textures, geometry, rendering buffers and other large assets. They complement each other rather than replacing each other.

d)Proposed Hierarchy

CPU/GPU → L1 → L2 → Large Shared L3 → High-Bandwidth DRAM

The design should use efficient cache policies, prefetching, high-bandwidth DRAM, asset compression and predictive streaming.

e)Causes of Frame Drops

- High DRAM utilization
- Cache misses
- Large texture transfers
- Slow asset streaming
- Poor access patterns
- CPU/GPU synchronization delays
- Insufficient memory bandwidth

f)Bandwidth Calculation

Texture data = 200 GB/s, geometry data = 100 GB/s, other rendering data = 50 GB/s.

Total requirement = 350 GB/s.

Available bandwidth = 448 GB/s.

Utilization = $(350/448) \times 100 = 78.1\%$.

Remaining headroom = 21.9%.

The headroom can absorb additional workload, but sustained increases in asset size may require further bandwidth optimization.

g)Proposed Improvements

1. Improve L3 cache efficiency.
2. Use high-bandwidth DRAM.

3. Prefetch data before it is required.
4. Compress textures and assets.
5. Stream assets according to player movement.
6. Optimize memory access patterns.
7. Reduce CPU/GPU synchronization overhead.

h)Advantages

- Reduced frame drops
- Faster scene loading
- Better texture streaming
- Improved CPU/GPU utilization
- Reduced memory stalls
- More stable frame rates

i)Conclusion

L3 cache minimizes latency for frequently reused data while high-bandwidth DRAM handles large game assets. Combining cache optimization, prefetching, compression and efficient streaming provides a practical solution for high-performance gaming.

j)Gaming Memory Comparison

Parameter	L3 Cache	DRAM
Technology	SRAM	DRAM
Latency	Very low	Higher
Capacity	Small	Large
Cost per bit	High	Lower
Main role	Frequently reused data	Large game assets
Performance benefit	Reduces CPU stalls	Provides high data bandwidth

k)Gaming Bandwidth Analysis

Data Type	Bandwidth	Percentage of 350 GB/s Workload
Textures	200 GB/s	57.1%
Geometry	100 GB/s	28.6%
Other rendering	50 GB/s	14.3%
Total	350 GB/s	100%

